

尚硅谷大数据技术之 SparkSQL

（作者：尚硅谷研究院）

版本：V5.0

第 1 章 Spark SQL 概述

1.1 什么是 Spark SQL

Spark SQL 是用于结构化数据处理的 Spark 模块。与基本的 Spark RDD API 不同，Spark SQL 提供的接口为 Spark 提供了有关数据结构和正在执行的计算的更多信息。在内部，Spark SQL 使用这些额外的信息来执行额外的优化。与 Spark SQL 交互的方式有多种，包括 SQL 和 Dataset API。计算结果时，使用相同的执行引擎，与您用于表达计算的 API/语言无关。

1.2 为什么要有 Spark SQL



1.2 为什么要有Spark SQL

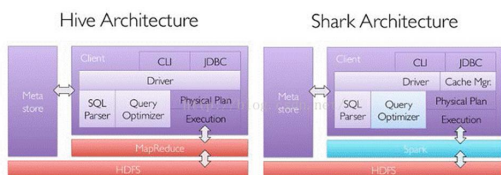


为了实现类似hive的功能

Hadoop	=> MapReduce	=> Hive (HQL)	
Spark	=> RDD	=> Shark (Spark + Hive)	=> Spark SQL
			=> Hive On Spark

Spark on Hive: 使用spark操作Hive中的表格Hive。只作为存储元数据，Spark负责SQL解析优化，语法是Spark SQL语法，Spark底层采用优化后的d或者ds执行。

Hive on Spark: 使用Hive，只是将原有的MR引擎替换为spark。Hive既作为存储元数据又负责SQL的解析优化，语法是HQL语法，执行引擎变成了Spark，Spark负责采用RDD执行。



让天下没有难学的技术

1.3 SparkSQL 的发展

1) 发展历史

RDD (Spark1.0) => Dataframe (Spark1.3) => Dataset (Spark1.6)

如果同样的数据都给到这三个数据结构，他们分别计算之后，都会给出相同的结果。不同的是他们的执行效率和执行方式。在现在的版本中，dataSet 性能最好，已经成为了唯一使用的接口。其中 Dataframe 已经在底层被看做是特殊泛型的 DataSet<Row>。

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：尚硅谷官网

2) 三者的共性

(1) RDD、DataFrame、DataSet 全都是 Spark 平台下的 **分布式弹性数据集**，为处理超大型数据提供便利。

(2) 三者都有惰性机制，在进行创建、转换，如 `map` 方法时，不会立即执行，**只有在遇到 Action 行动算子如 `foreach` 时，三者才会开始遍历运算。**

(3) 三者有许多 **共同的函数**，如 `filter`，排序等。

(4) 三者都会根据 Spark 的内存情况 **自动缓存运算。**

(5) 三者都有 **分区** 的概念。

1.4 Spark SQL 的特点

1) 易整合

无缝的整合了 SQL 查询和 Spark 编程。

Integrated

Seamlessly mix SQL queries with Spark programs.

Spark SQL lets you query structured data inside Spark programs, using either SQL or a familiar [DataFrame API](#). Usable in Java, Scala, Python and R.

```
context = HiveContext(sc)
results = context.sql(
  "SELECT * FROM people")
names = results.map(lambda p: p.name)
```

Apply functions to results of SQL queries.

2) 统一的数据访问方式

使用相同的方式连接不同的数据源。

Uniform Data Access

Connect to any data source the same way.

DataFrames and SQL provide a common way to access a variety of data sources, including Hive, Avro, Parquet, ORC, JSON, and JDBC. You can even join data across these sources.

```
context.jsonFile("s3n://...")
  .registerTempTable("json")
results = context.sql(
  """SELECT *
  FROM people
  JOIN json ...""")
```

Query and join different data sources.

3) 兼容 Hive

在已有的仓库上直接运行 SQL 或者 HQL。

Hive Compatibility

Run unmodified Hive queries on existing data.

Spark SQL reuses the Hive frontend and metastore, giving you full compatibility with existing Hive data, queries, and UDFs. Simply install it alongside Hive.

Meta Store	HiveQL	UDFs	SerDes
Spark SQL			
Apache Spark			

Spark SQL can use existing Hive metastores, SerDes, and UDFs.

4) 标准的数据连接

通过 JDBC 或者 ODBC 来连接。



第 2 章 Spark SQL 编程

2.1 SparkSession 新的起始点

在老的版本中，SparkSQL 提供两种 SQL 查询起始点：

- 一个叫 `SQLContext`，用于 Spark 自己提供的 SQL 查询；
- 一个叫 `HiveContext`，用于连接 Hive 的查询。

SparkSession 是 Spark 最新的 SQL 查询起始点，实质上是 SQLContext 和 HiveContext 的组合，所以在 SQLContext 和 HiveContext 上可用的 API 在 SparkSession 上同样是可以使用的。

SparkSession 内部封装了 SparkContext，所以计算实际上是由 SparkContext 完成的。当我们使用 spark-shell 的时候，Spark 框架会自动的创建一个名称叫做 Spark 的 SparkSession，就像我们以前可以自动获取到一个 sc 来表示 SparkContext。

```
[atguigu@hadoop102 spark-local]$ bin/spark-shell
```

20/09/12 11:16:35 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at <http://hadoop102:4040>
Spark context available as 'sc' (master = local[*], app id = local-1599880621394).
Spark session available as 'spark'.
Welcome to

```

/ _/ _ _ _ _// _
 \V \V \ ' _/ ' _/
/_/_/. _ _ ,/_/_/_/_ \ version 3.3.1
/_/_/

```

Using Scala version 2.12.10 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_212)
Type in expressions to have them evaluated.
Type :help for more information.

2.2 常用方式

2.2.1 方法调用

- 1) 创建一个 maven 工程 SparkSQL
- 2) 创建包名为 com.atguigu.sparksql
- 3) 输入文件夹准备：在新建的 SparkSQL 项目名称上右键=》新建 input 文件夹=》在 input 文件夹上右键=》新建 user.json。并输入如下内容：

```
{ "age":20,"name":"qiaofeng" }
{ "age":19,"name":"xuzhu" }
{ "age":18,"name":"duanyu" }
{ "age":22,"name":"qiaofeng" }
{ "age":11,"name":"xuzhu" }
{ "age":12,"name":"duanyu" }
```

- 5) 在 pom.xml 文件中添加 spark-sql 的依赖

```
<dependencies>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-sql_2.12</artifactId>
    <version>3.3.1</version>
  </dependency>

  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.22</version>
  </dependency>
</dependencies>
```

- 6) 代码实现

添加 javaBean 的 User

```
package com.atguigu.sparksql.Bean;

import lombok.Data;
import java.io.Serializable;

@Data
public class User implements Serializable {
    public Long age;
    public String name;

    public User() {
    }

    public User(Long age, String name) {
        this.age = age;
        this.name = name;
    }
}
```

}

代码编写

```
package com.atguigu.sparksql;

import com.atguigu.sparksql.Bean.User;
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.function.MapFunction;
import org.apache.spark.api.java.function.ReduceFunction;
import org.apache.spark.sql.*;
import scala.Tuple2;

public class Test01_Method {
    public static void main(String[] args) {

        //1. 创建配置对象
        SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");

        //2. 获取 sparkSession
        SparkSession spark = SparkSession.builder().config(conf).getOrCreate();

        //3. 编写代码
        // 按照行读取
        Dataset<Row> lineDS = spark.read().json("input/user.json");

        // 转换为类和对象
        Dataset<User> userDS = lineDS.as(Encoders.bean(User.class));

        // userDS.show();

        // 使用方法操作
        // 函数式的方法
        Dataset<User> userDataset = lineDS.map(new MapFunction<Row, User>() {
            @Override
            public User call(Row value) throws Exception {
                return new User(value.getLong(0), value.getString(1));
            }
        },
        // 使用 kryo 在底层会有部分算子无法使用
        Encoders.bean(User.class));

        // 常规方法
        Dataset<User> sortDS = userDataset.sort(new Column("age"));
        sortDS.show();

        // 区分
        RelationalGroupedDataset groupByDS = userDataset.groupBy("name");

        // 后续方法不同
        Dataset<Row> count = groupByDS.count();

        // 推荐使用函数式的方法 使用更灵活
        KeyValueGroupedDataset<String, User> groupedDataset = userDataset.groupByKey(new
        MapFunction<User, String>() {
```

```
@Override
public String call(User value) throws Exception {

    return value.name;
}
}, Encoders.STRING());

// 聚合算子都是从 groupByKey 开始
// 推荐使用 reduceGroup
Dataset<Tuple2<String, User>> result = groupedDataset.reduceGroups(new
ReduceFunction<User>() {
    @Override
    public User call(User v1, User v2) throws Exception {
        // 取用户的大年龄
        return new User(Math.max(v1.age, v2.age), v1.name);
    }
});

result.show();

//4. 关闭 sparkSession
spark.close();
}
}
```

在 sparkSql 中 DS 直接支持的转换算子有：map（底层已经优化为 mapPartition）、mapPartition、flatMap、groupByKey（聚合算子全部由 groupByKey 开始）、filter、distinct、coalesce、repartition、**sort** 和 **orderBy**（不是函数式的算子，不过不影响使用）。

2.2.2 SQL 使用方式

```
package com.atguigu.sparksql;

import org.apache.spark.SparkConf;
import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.SparkSession;

public class Test02_SQL {
    public static void main(String[] args) {

        //1. 创建配置对象
        SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");

        //2. 获取 sparkSession
        SparkSession spark = SparkSession.builder().config(conf).getOrCreate();

        //3. 编写代码
        Dataset<Row> lineDS = spark.read().json("input/user.json");

        // 创建视图 => 转换为表格 填写表名
        // 临时视图的生命周期和当前的 sparkSession 绑定
        // orReplace 表示覆盖之前相同名称的视图
```

```
lineDS.createOrReplaceTempView("t1");

// 支持所有的 hive sql 语法,并且会使用 spark 的又花钱
Dataset<Row> result = spark.sql("select * from t1 where age > 18");

result.show();

//4. 关闭 sparkSession
spark.close();
}
}}
```

2.2.3 DSL 特殊语法（扩展）

```
package com.atguigu.sparksql;

import org.apache.spark.SparkConf;
import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.SparkSession;

import static org.apache.spark.sql.functions.col;

public class Test03_DSL {
    public static void main(String[] args) {
        //1. 创建配置对象
        SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");

        //2. 获取 sparkSession
        SparkSession spark = SparkSession.builder().config(conf).getOrCreate();

        //3. 编写代码
        // 导入特殊的依赖 import static org.apache.spark.sql.functions.col;
        Dataset<Row> lineRDD = spark.read().json("input/user.json");

        Dataset<Row> result =
        lineRDD.select(col("name").as("newName"),col("age").plus(1).as("newAge"))
                .filter(col("age").gt(18));

        result.show();

        //4. 关闭 sparkSession
        spark.close();
    }
}
```

2.3 SQL 语法的用户自定义函数

2.3.1 UDF

1) UDF：一行进入，一行出

2) 代码实现

```
package com.atguigu.sparksql;
```

```
import org.apache.spark.SparkConf;
import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.SparkSession;
import org.apache.spark.sql.api.java.UDF1;
import org.apache.spark.sql.expressions.UserDefinedFunction;
import org.apache.spark.sql.types.DataTypes;

import static org.apache.spark.sql.functions.udf;

public class Test04_UDF {
    public static void main(String[] args) {

        //1. 创建配置对象
        SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");

        //2. 获取 sparkSession
        SparkSession spark = SparkSession.builder().config(conf).getOrCreate();

        //3. 编写代码
        Dataset<Row> lineRDD = spark.read().json("input/user.json");

        lineRDD.createOrReplaceTempView("user");

        // 定义一个函数
        // 需要首先导入依赖 import static org.apache.spark.sql.functions.udf;
        UserDefinedFunction addName = udf(new UDF1<String, String>() {
            @Override
            public String call(String s) throws Exception {
                return s + " 大侠";
            }
        }, DataTypes.StringType);

        spark.udf().register("addName",addName);

        spark.sql("select addName(name) newName from user")
            .show();

        // lambda 表达式写法
        spark.udf().register("addName1",(UDF1<String,String>) name -> name + " 大侠",DataTypes.StringType);

        //4. 关闭 sparkSession
        spark.close();
    }
}
```

2.3.2 UDAF

1) UDAF: 输入多行, 返回一行。通常和 `groupBy` 一起使用, 如果直接使用 UDAF 函数, 默认将所有的数据合并在一起。

- 2) Spark3.x 推荐使用 extends Aggregator 自定义 UDAF，属于强类型的 Dataset 方式。
- 3) Spark2.x 使用 extends UserDefinedAggregateFunction，属于弱类型的 DataFrame
- 4) 案例实操

需求：实现求平均年龄，自定义 UDAF，MyAvg(age)

(1) 自定义聚合函数实现-强类型

```
package com.atguigu.sparksql;

import org.apache.spark.SparkConf;
import org.apache.spark.sql.Encoder;
import org.apache.spark.sql.Encoders;
import org.apache.spark.sql.Session;
import org.apache.spark.sql.expressions.Aggregator;

import java.io.Serializable;

import static org.apache.spark.sql.functions.udaf;

public class Test05_UDAF {
    public static void main(String[] args) {

        //1. 创建配置对象
        SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");

        //2. 获取 sparkSession
        Session spark = Session.builder().config(conf).getOrCreate();

        //3. 编写代码
        spark.read().json("input/user.json").createOrReplaceTempView("user");

        // 注册需要导入依赖 import static org.apache.spark.sql.functions.udaf;
        spark.udf().register("avgAge",udaf(new MyAvg(),Encoders.LONG()));

        spark.sql("select avgAge(age) newAge from user").show();

        //4. 关闭 sparkSession
        spark.close();
    }

    public static class Buffer implements Serializable {
        private Long sum;
        private Long count;

        public Buffer() {
        }

        public Buffer(Long sum, Long count) {
            this.sum = sum;
            this.count = count;
        }
    }
}
```

```
public Long getSum() {
    return sum;
}

public void setSum(Long sum) {
    this.sum = sum;
}

public Long getCount() {
    return count;
}

public void setCount(Long count) {
    this.count = count;
}
}

public static class MyAvg extends Aggregator<Long,Buffer,Double>{

    @Override
    public Buffer zero() {
        return new Buffer(0L,0L);
    }

    @Override
    public Buffer reduce(Buffer b, Long a) {
        b.setSum(b.getSum() + a);
        b.setCount(b.getCount() + 1);
        return b;
    }

    @Override
    public Buffer merge(Buffer b1, Buffer b2) {

        b1.setSum(b1.getSum() + b2.getSum());
        b1.setCount(b1.getCount() + b2.getCount());

        return b1;
    }

    @Override
    public Double finish(Buffer reduction) {
        return reduction.getSum().doubleValue() / reduction.getCount();
    }

    @Override
    public Encoder<Buffer> bufferEncoder() {
        // 可以用 kryo 进行优化
        return Encoders.kryo(Buffer.class);
    }

    @Override
    public Encoder<Double> outputEncoder() {
        return Encoders.DOUBLE();
    }
}
```

```
}  
}
```

2.3.3 UDTF（没有）

输入一行，返回多行（Hive）。

SparkSQL 中没有 UDTF，需要使用算子类型的 flatMap 先完成拆分。

第 3 章 SparkSQL 数据的加载与保存

3.1 读取和保存文件

SparkSQL 读取和保存的文件一般为三种，JSON 文件、CSV 文件和列式存储的文件，同时可以通过添加参数，来识别不同的存储和压缩格式。

3.1.1 CSV 文件

1) 代码实现

```
package com.atguigu.sparksql;  
  
import com.atguigu.sparksql.Bean.User;  
import org.apache.spark.SparkConf;  
import org.apache.spark.api.java.function.MapFunction;  
import org.apache.spark.sql.*;  
  
public class Test06_CSV {  
    public static void main(String[] args) throws ClassNotFoundException {  
  
        //1. 创建配置对象  
        SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");  
  
        //2. 获取 sparkSession  
        SparkSession spark = SparkSession.builder().config(conf).getOrCreate();  
  
        //3. 编写代码  
        DataFrameReader reader = spark.read();  
  
        // 添加参数 读取 csv  
        Dataset<Row> userDS = reader  
            .option("header", "true")//默认为 false 不读取列名  
            .option("sep", ",") // 默认为, 列的分割  
            // 不需要写压缩格式 自适应  
            .csv("input/user.csv");  
  
        userDS.show();  
  
        // 转换为 user 的 ds  
        // 直接转换类型会报错 csv 读取的数据都是 string  
        Dataset<User> userDS1 = userDS.as(Encoders.bean(User.class));  
        userDS1.printSchema();  
    }  
}
```

```
Dataset<User> userDS1 = userDS.map(new MapFunction<Row, User>() {
    @Override
    public User call(Row value) throws Exception {
        return new User(Long.valueOf(value.getString(0)), value.getString(1));
    }
}, Encoders.bean(User.class));

userDS1.show();

// 写出为 csv 文件
DataFrameWriter<User> writer = userDS1.write();

writer.option("seq", ";")
    .option("header", "true")
//    .option("compression", "gzip")// 压缩格式
// 写出模式
// append 追加
// Ignore 忽略本次写出
// Overwrite 覆盖写
// ErrorIfExists 如果存在报错
    .mode(SaveMode.Append)
    .csv("output");

//4. 关闭 sparkSession
spark.close();
}
```

3.1.2 JSON 文件

```
package com.atguigu.sparksql;

import com.atguigu.sparksql.Bean.User;
import org.apache.spark.SparkConf;
import org.apache.spark.sql.*;

public class Test07_JSON {
    public static void main(String[] args) {
        //1. 创建配置对象
        SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");

        //2. 获取 sparkSession
        SparkSession spark = SparkSession.builder().config(conf).getOrCreate();

        //3. 编写代码
        Dataset<Row> json = spark.read().json("input/user.json");

        // json 数据可以读取数据的数据类型
        Dataset<User> userDS = json.as(Encoders.bean(User.class));

        userDS.show();

        // 读取别的类型的数据也能写出为 json
    }
}
```

```
    DataFrameWriter<User> writer = userDS.write();

    writer.json("output1");

    //4. 关闭 sparkSession
    spark.close();
}
}
```

3.1.3 Parquet 文件

列式存储的数据自带列分割。

```
package com.atguigu.sparksql;

import org.apache.spark.SparkConf;
import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.SparkSession;

public class Test08_Parquet {
    public static void main(String[] args) {

        //1. 创建配置对象
        SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");

        //2. 获取 sparkSession
        SparkSession spark = SparkSession.builder().config(conf).getOrCreate();

        //3. 编写代码
        Dataset<Row> json = spark.read().json("input/user.json");

        // 写出默认使用 snappy 压缩
        json.write().parquet("output");

        // 读取 parquet 自带解析 能够识别列名
        Dataset<Row> parquet = spark.read().parquet("output");

        parquet.printSchema();

        //4. 关闭 sparkSession
        spark.close();
    }
}
```

3.2 与 MySQL 交互

1) 导入依赖

```
<!-- 早期 MySQL 版本 -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.27</version>
</dependency>
<!-- 5.8(8.0)MySQL 版本 -->
```

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.18</version>
</dependency>
```

2) 从 MySQL 读数据

```
package com.atguigu.sparksql;

import org.apache.spark.SparkConf;
import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.SparkSession;

import java.util.Properties;

public class Test09_Table {
    public static void main(String[] args) {

        //1. 创建配置对象
        SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");

        //2. 获取 sparkSession
        SparkSession spark = SparkSession.builder().config(conf).getOrCreate();

        //3. 编写代码
        Dataset<Row> json = spark.read().json("input/user.json");

        // 添加参数
        Properties properties = new Properties();
        properties.setProperty("user", "root");
        properties.setProperty("password", "000000");

        //      json.write()
        //      // 写出模式针对于表格追加覆盖
        //      .mode(SaveMode.Append)
        //      jdbc("jdbc:mysql://hadoop102:3306", "gmall.testInfo", properties);

        Dataset<Row>          jdbc
        spark.read().jdbc("jdbc:mysql://hadoop102:3306/gmall?useSSL=false&useUnicode=true&characterEncoding=UTF-8&allowPublicKeyRetrieval=true", "test info", properties);

        jdbc.show();

        //4. 关闭 sparkSession
        spark.close();
    }
}
```

3.3 与 Hive 交互

SparkSQL 可以采用内嵌 Hive（spark 开箱即用的 hive），也可以采用外部 Hive。企业开发中，通常采用外部 Hive。

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

3.3.1 Linux 中的交互

1) 添加 MySQL 连接驱动到 spark-yarn 的 jars 目录

```
[atguigu@hadoop102 spark-yarn]$ cp /opt/software/mysql-connector-java-5.1.27-bin.jar /opt/module/spark-yarn/jars
```

2) 添加 hive-site.xml 文件到 spark-yarn 的 conf 目录

```
[atguigu@hadoop102 spark-yarn]$ cp /opt/module/hive/conf/hive-site.xml /opt/module/spark-yarn/conf
```

3) 启动 spark-sql 的客户端即可

```
[atguigu@hadoop102 spark-yarn]$ bin/spark-sql --master yarn
spark-sql (default)> show tables;
```

3.3.2 IDEA 中的交互

1) 添加依赖

```
<dependencies>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-sql 2.12</artifactId>
    <version>3.3.1</version>
  </dependency>

  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.27</version>
  </dependency>

  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-hive_2.12</artifactId>
    <version>3.3.1</version>
  </dependency>

  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.22</version>
  </dependency>
</dependencies>
```

2) 拷贝 **hive-site.xml** 到 resources 目录（如果需要操作 Hadoop，需要拷贝 hdfs-site.xml、core-site.xml、yarn-site.xml）

3) 代码实现

```
package com.atguigu.sparksql;

import org.apache.spark.SparkConf;
import org.apache.spark.sql.SparkSession;

public class Test10_Hive {
```

```
public static void main(String[] args) {  
    System.setProperty("HADOOP_USER_NAME","atguigu");  
  
    //1. 创建配置对象  
    SparkConf conf = new SparkConf().setAppName("sparksql").setMaster("local[*]");  
  
    //2. 获取 sparkSession  
    SparkSession spark = SparkSession.builder()  
        .enableHiveSupport()// 添加 hive 支持  
        .config(conf).getOrCreate();  
  
    //3. 编写代码  
    spark.sql("show tables").show();  
  
    spark.sql("create table user_info(name String,age bigint)");  
    spark.sql("insert into table user_info values('zhangsan',10)");  
    spark.sql("select * from user_info").show();  
  
    //4. 关闭 sparkSession  
    spark.close();  
}  
}
```

第 4 章 SparkSQL 项目实战

4.1 准备数据

我们这次 Spark-sql 操作所有的数据均来自 Hive，首先在 Hive 中创建表，并导入数据。

一共有 3 张表：1 张用户行为表，1 张城市表，1 张产品表。

1) 将 city_info.txt、product_info.txt、user_visit_action.txt 上传到/opt/module/data

```
[atguigu@hadoop102 module]$ mkdir data
```

2) 将创建对应的三张表

```
hive (default)>  
CREATE TABLE `user_visit_action`(  
    `date` string,  
    `user_id` bigint,  
    `session_id` string,  
    `page_id` bigint,  
    `action_time` string,  
    `search_keyword` string,  
    `click_category_id` bigint,  
    `click_product_id` bigint, --点击商品 id，没有商品用-1 表示。  
    `order_category_ids` string,  
    `order_product_ids` string,  
    `pay_category_ids` string,  
    `pay_product_ids` string,  
    `city_id` bigint --城市 id  
)  
row format delimited fields terminated by '\t';
```



```
CREATE TABLE `city_info`(  
  `city_id` bigint, --城市 id  
  `city_name` string, --城市名称  
  `area` string --区域名称  
)  
row format delimited fields terminated by '\t';
```

```
CREATE TABLE `product_info`(  
  `product_id` bigint, -- 商品 id  
  `product_name` string, --商品名称  
  `extend_info` string  
)  
row format delimited fields terminated by '\t';
```

3) 并加载数据

```
hive (default)>  
load data local inpath '/opt/module/data/user_visit_action.txt' into table user_visit_action;  
load data local inpath '/opt/module/data/product_info.txt' into table product_info;  
load data local inpath '/opt/module/data/city_info.txt' into table city_info;
```

4) 测试一下三张表数据是否正常

```
hive (default)>  
select * from user_visit_action limit 5;  
select * from product_info limit 5;  
select * from city_info limit 5;
```

4.2 需求：各区域热门商品 Top3

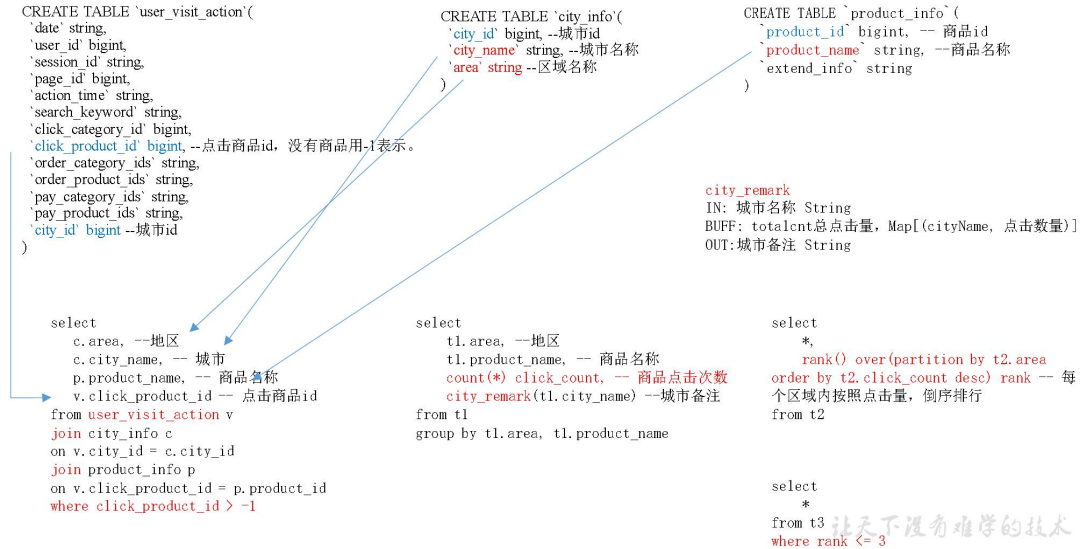
4.2.1 需求简介

这里的热门商品是从**点击量**的维度来看的，计算各个区域前三大热门商品，并备注上每个商品在主要城市中的分布比例，超过两个城市用其他显示。

例如：

地区	商品名称	点击次数	城市备注
华北	商品 A	100000	北京 21.2%，天津 13.2%，其他 65.6%
华北	商品 P	80200	北京 63.0%，太原 10%，其他 27.0%
华北	商品 M	40000	北京 63.0%，太原 10%，其他 27.0%
东北	商品 J	92000	大连 28%，辽宁 17.0%，其他 55.0%

4.2.2 思路分析



使用 Spark-SQL 来完成复杂的需求, 可以使用 UDF 或 UDAF。

- (1) 查询出来所有的点击记录, 并与 city_info 表连接, 得到每个城市所在的地区, 与 Product_info 表连接得到商品名称。
- (2) 按照地区和商品名称分组, 统计出每个商品在每个地区的总点击次数。
- (3) 每个地区内按照点击次数降序排列。
- (4) 只取前三名, 并把结果保存在数据库中。
- (5) 城市备注需要自定义 UDAF 函数。

4.2.3 代码实现

```

package com.atguigu.sparksql.demo;

import lombok.Data;
import org.apache.spark.SparkConf;
import org.apache.spark.sql.*;
import org.apache.spark.sql.expressions.Aggregator;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.TreeMap;
import java.util.function.BiConsumer;
import static org.apache.spark.sql.functions.udaf;

public class Test01_Top3 {
    public static void main(String[] args) {
  
```

```
// 1. 创建 sparkConf 配置对象
SparkConf conf = new SparkConf().setAppName("sql").setMaster("local[*]");

// 2. 创建 sparkSession 连接对象
SparkSession spark =
SparkSession.builder().enableHiveSupport().config(conf).getOrCreate();

// 3. 编写代码
// 将 3 个表格数据 join 在一起
Dataset<Row> t1DS = spark.sql("select \n" +
    "\tc.area,\n" +
    "\tc.city_name,\n" +
    "\tp.product_name\n" +
    "from\n" +
    "\tuser_visit_action u\n" +
    "join\n" +
    "\tcity_info c\n" +
    "on\n" +
    "\tu.city_id=c.city_id\n" +
    "join\n" +
    "\tp.product_info p\n" +
    "on\n" +
    "\tu.click_product_id=p.product_id");

t1DS.createOrReplaceTempView("t1");

spark.udf().register("cityMark", udaf(new CityMark()), Encoders.STRING());

// 将区域内的产品点击次数统计出来
Dataset<Row> t2ds = spark.sql("select \n" +
    "\tarea,\n" +
    "\tproduct_name,\n" +
    "\tcityMark(city_name) mark,\n" +
    "\tcount(*) counts\n" +
    "from\n" +
    "\tt1\n" +
    "group by\n" +
    "\tarea,product_name");

// t2ds.show(false);
t2ds.createOrReplaceTempView("t2");

// 对区域内产品点击的次数进行排序 找出区域内的 top3
spark.sql("select\n" +
    "\tarea,\n" +
    "\tproduct_name,\n" +
    "\tmark,\n" +
    "\trank() over (partition by area order by counts desc) rk\n" +
    "from\n" +
    "\tt2").createOrReplaceTempView("t3");

// 使用过滤 取出区域内的 top3
spark.sql("select\n" +
    "\tarea,\n" +
    "\tproduct_name,
```

```
"\tmark \n" +
"from\n" +
"\tt3\n" +
"where \n" +
"\trk < 4").show(50,false);

// 4. 关闭 sparkSession
spark.close();
}

@Data
public static class Buffer implements Serializable {
    private Long totalCount;
    private HashMap<String,Long> map;

    public Buffer() {
    }

    public Buffer(Long totalCount, HashMap<String, Long> map) {
        this.totalCount = totalCount;
        this.map = map;
    }
}

public static class CityMark extends Aggregator<String, Buffer, String> {

    public static class CityCount {
        public String name;
        public Long count;

        public CityCount(String name, Long count) {
            this.name = name;
            this.count = count;
        }

        public CityCount() {
        }

    }

}

public static class CompareCityCount implements Comparator<CityCount> {

    /**
     * 默认倒序
     * @param o1
     * @param o2
     * @return
     */
    @Override
    public int compare(CityCount o1, CityCount o2) {
        if (o1.count > o2.count) {
            return -1;
        } else return o1.count.equals(o2.count) ? 0 : 1;
    }
}
```

```
}

@Override
public Buffer zero() {
    return new Buffer(0L, new HashMap<String, Long>());
}

/**
 * 分区内的预聚合
 *
 * @param b map(城市,sum)
 * @param a 当前行表示的城市
 * @return
 */
@Override
public Buffer reduce(Buffer b, String a) {
    HashMap<String, Long> hashMap = b.getMap();
    // 如果 map 中已经有当前城市 次数+1
    // 如果 map 中没有当前城市 0+1
    hashMap.put(a, hashMap.getOrDefault(a, 0L) + 1);

    b.setTotalCount(b.getTotalCount() + 1L);
    return b;
}

/**
 * 合并多个分区的数据
 *
 * @param b1 (北京,100),(上海,200)
 * @param b2 (天津,100),(上海,200)
 * @return
 */
@Override
public Buffer merge(Buffer b1, Buffer b2) {
    b1.setTotalCount(b1.getTotalCount() + b2.getTotalCount());

    HashMap<String, Long> map1 = b1.getMap();
    HashMap<String, Long> map2 = b2.getMap();
    // 将 map2 中的数据放入合并到 map1
    map2.forEach(new BiConsumer<String, Long>() {
        @Override
        public void accept(String s, Long aLong) {
            map1.put(s, aLong + map1.getOrDefault(s, 0L));
        }
    });

    return b1;
}

/**
 * map => {(上海,200),(北京,100),(天津,300)}
 *
 * @param reduction
 * @return
 */
```

```
*/
@Override
public String finish(Buffer reduction) {
    Long totalCount = reduction.getTotalCount();
    HashMap<String, Long> map = reduction.getMap();
    // 需要对 map 中的 value 次数进行排序
    ArrayList<CityCount> cityCounts = new ArrayList<>();

    // 将 map 中的数据放入到 treeMap 中 进行排序
    map.forEach(new BiConsumer<String, Long>() {
        @Override
        public void accept(String s, Long aLong) {
            cityCounts.add(new CityCount(s, aLong));
        }
    });

    cityCounts.sort(new CompareCityCount());
    ArrayList<String> resultMark = new ArrayList<>();

    Double sum = 0.0;

    // 当前没有更多的城市数据 或者 已经找到两个城市数据了 停止循环
    while (!(cityCounts.size() == 0) && resultMark.size() < 2) {
        CityCount cityCount = cityCounts.get(0);
        resultMark.add(cityCount.name + String.format("%.2f", cityCount.count.doubleValue() /
totalCount * 100) + "%");
        cityCounts.remove(0);
    }

    // 拼接其他城市
    if (cityCounts.size() > 0) {
        resultMark.add("其他" + String.format("%.2f", 100 - sum) + "%");
    }

    StringBuilder cityMark = new StringBuilder();
    for (String s : resultMark) {
        cityMark.append(s).append(",");
    }

    return cityMark.substring(0, cityMark.length() - 1);
}

@Override
public Encoder<Buffer> bufferEncoder() {
    return Encoders.javaSerialization(Buffer.class);
}

@Override
public Encoder<String> outputEncoder() {
    return Encoders.STRING();
}
}
```